Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 6

Design and Implement an API Rate Limiter using Token Bucket

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198
Vino S R Harrison	3122 24 5001 199

Design and Implement an API Rate Limiter using Token Bucket

Aim

- To implement a Rate Limiter using the Token Bucket algorithm in a Spring Boot application, using Redis to maintain the rate-limit information across instances.
- To analyse request control under normal and burst traffic, and to prove that concurrent requests are handled correctly using an atomic Redis Lua script.

Question

A web application may receive a large number of API requests from users, which can overload backend services. Design and implement a Rate Limiter to restrict the number of requests from each client using the Token Bucket algorithm, storing token information in Redis. The implementation must:

1) Develop a REST API using Spring Boot.

2) Configure Redis with the Spring Boot application.

3) Identify clients using a client ID or IP address.

4) Implement the Token Bucket algorithm with configurable bucket capacity and refill rate.

5) Store the token count and last refill time in Redis.

6) Allow requests when tokens are available and return HTTP 429 Too Many Requests when the limit is exceeded.

7) Generate normal and burst requests and observe the rate-limiting behaviour.

8) Test concurrent requests and use Redis Lua scripting for atomic token updates.

Procedure (Workflow)

1) A Redis 7 container is started with Docker Compose and exposed on localhost:6379.

2) The Spring Boot application starts and connects to Redis via spring-boot-starter-data-redis (StringRedisTemplate / Lettuce).

3) Each incoming request is tagged with a client identity resolved in priority order: the X-Client-Id header, the clientId query parameter, or the caller's IP address.

4) The client's bucket state {tokens, ts} is kept in Redis under the key ratelimit:<clientId>; a first visit initialises a full bucket of 'capacity' tokens.

5) Every request executes one atomic Lua script (EVAL token_bucket.lua): it reads the bucket, refills tokens from the elapsed time, and consumes one token if available.

6) If a token was available the API returns HTTP 200 with the remaining tokens; otherwise it returns HTTP 429 Too Many Requests with a Retry-After header.

7) A Python traffic driver generates NORMAL traffic (10 requests at 250 ms intervals, i.e. below the 5 tokens/s refill), BURST traffic (50 simultaneous requests), and a DRAIN (sequential requests until the first 429).

8) An in-app concurrency endpoint (/api/test/concurrent) fires 20 threads x 20 requests at one shared bucket to verify that the atomic Lua update never overspends.

9) Bucket state is inspected with GET /api/bucket/{clientId} and directly in Redis (redis-cli HGETALL) before and after the runs.

10) Observed results are charted and written into this report, and 33 unit + integration tests verify the algorithm (Lua path tested when Redis is reachable).

System Architecture & Workflow

The following diagram illustrates the token-bucket rate limiter architecture. Clients (identified by client ID or IP) reach the Spring Boot API. Every request asks a shared Redis for one token; because the refill/consume runs inside a single Lua EVAL, any number of application instances can share the same buckets safely.

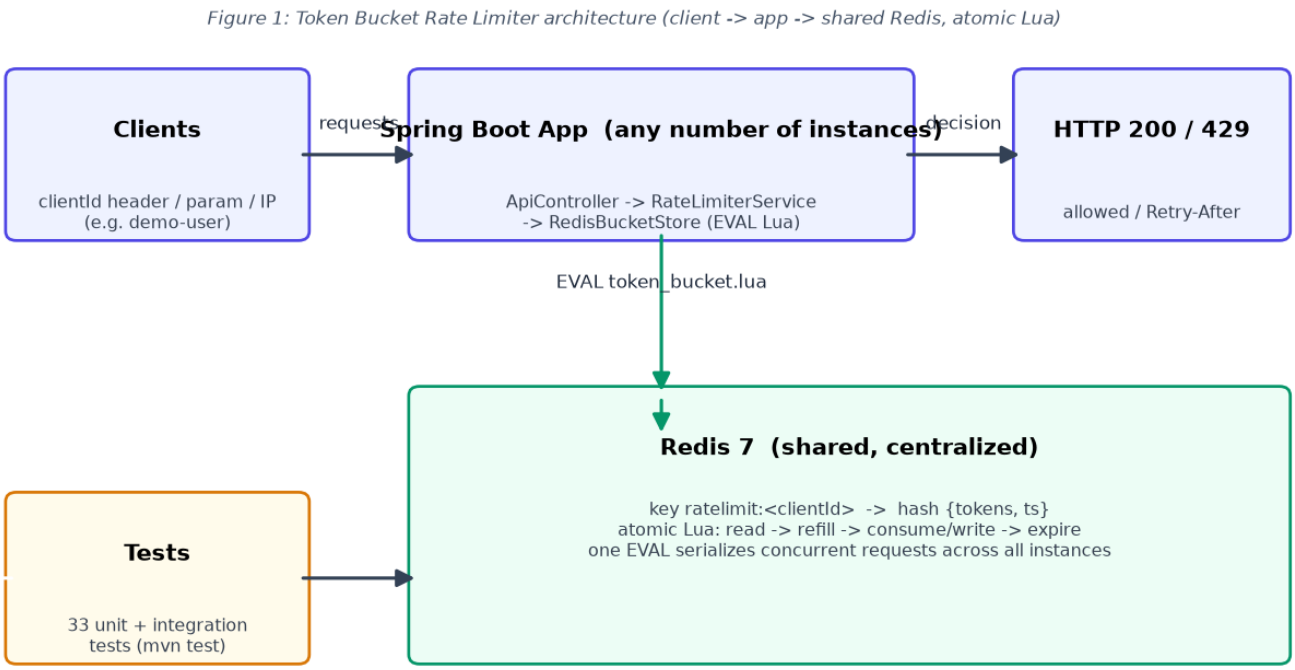


Figure 1: Token Bucket Rate Limiter Architecture (shared Redis + atomic Lua)

Design of the Token Bucket Rate Limiter

1. Token Bucket Algorithm (step-by-step)

```
for each request R from client C:
  (now, tokens, ts) = read bucket {tokens, ts} of C from Redis
                      (absent -> tokens = capacity, ts = now)

  # lazy refill from elapsed time
  elapsedSeconds = (now - ts) / 1000
  tokens         = min(capacity, tokens + elapsedSeconds * refillRate)

  # admission decision
  if tokens >= 1:
     tokens   = tokens - 1
     respond HTTP 200 with remaining = tokens
  else:
     respond HTTP 429 Too Many Requests (Retry-After = 1)

  # persist + stale-client cleanup (atomic, TTL refreshed)
```

2. Docker Compose Service Configuration

```
redis:
  image: redis:7-alpine
  command: ["redis-server", "--appendonly", "yes"]
  ports: ["6379:6379"]
  volumes: ["./redis-data:/data"]
```

3. Client Identification (header > query parameter > IP)

```
String header = request.getHeader("X-Client-Id");   // 1st priority
String param  = request.getParameter("clientId");    // 2nd priority
return header != null ? header : (param != null ? param
                        : request.getRemoteAddr()); // 3rd (IP)
```

4. Bucket State in Redis (hash {tokens, ts})

```
127.0.0.1:6379> HGETALL ratelimit:demo-user
1) "tokens"
2) "0.16000000000000042"
3) "ts"
4) "1786628433314"
```

5. Token Bucket Math (reference logic)

```
double refilled = Math.min(capacity, tokens + elapsedSeconds * tokensPerSecond);
boolean allowed = refilled >= 1.0;
double remaining = allowed ? refilled - 1.0 : refilled;
```

6. Atomic Token Update using Redis Lua (the core requirement)

The complete script below (src/main/resources/scripts/token_bucket.lua) is executed with EVAL. Redis serialises script execution, so concurrent requests - across threads and across instances - can never double-spend a token.

```
-- KEYS[1] = ratelimit:<clientId>   ARGV[1]=capacity ARGV[2]=tokens/s
-- ARGV[3]=now(ms) ARGV[4]=ttl(s). Hash {tokens, ts}.
local fields = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens, ts = tonumber(fields[1]), tonumber(fields[2])
if not tokens then tokens, ts = tonumber(ARGV[1]), tonumber(ARGV[3]) end
local refilled = tokens + (ARGV[3] - ts) / 1000.0 * tonumber(ARGV[2])
if refilled > tonumber(ARGV[1]) then refilled = tonumber(ARGV[1]) end
local allowed = 0
if refilled >= 1 then refilled = refilled - 1; allowed = 1 end
redis.call('HMSET', KEYS[1], 'tokens', refilled, 'ts', ARGV[3])
redis.call('EXPIRE', KEYS[1], ARGV[4])
-- encode: >=0 allowed (remaining*1000), <0 rejected
local rem = math.floor(refilled * 1000)
if allowed == 1 then return rem else return -rem - 1 end
```

7. HTTP 429 Response with Retry-After

```
HTTP/1.1 429
Retry-After: 1
X-RateLimit-Remaining: 0
{"status":429,"error":"Too Many Requests","message":"Rate limit exceeded for client 'demo'"}
```

8. Configurable Parameters (application.properties)

Bucket capacity, refill rate, key prefix, key TTL and the backend store are all configurable via application.properties (each also overridable with an environment variable):

```
ratelimit.capacity=${RATELIMIT_CAPACITY:10}           # max tokens per bucket (burst cap)
ratelimit.refill-per-second=${RATELIMIT_REFILL_PER_SECOND:5}  # sustained requests/second
ratelimit.prefix=${RATELIMIT_PREFIX:ratelimit}              # Redis key prefix
ratelimit.ttl-seconds=${RATELIMIT_TTL_SECONDS:3600}         # stale-client cleanup
ratelimit.store=${RATELIMIT_STORE:redis}                    # redis (Lua) or memory (test)
```

9. Redis Data Model

Key / Field	Type	Description
ratelimit:<clientId>	String key	Per-client bucket key (TTL for stale-client cleanup)
tokens	Number (double)	Current token count, refilled lazily from elapsed time
ts	Number (ms)	Last refill timestamp (epoch millis)

Output Results

All results below were observed in a live run: Redis 7 in Docker, Spring Boot on localhost:8080, capacity = 10 tokens, refill rate = 5 tokens/s. Every figure slot is filled by a real screenshot when present (assets/images/), otherwise by the chart generated from reports/.

0. Application Startup, Swagger and Tests

The application started on port 8080 and connected to Redis. The REST API is documented by Swagger UI; every protected endpoint is listed under /api, and each client is resolved by the X-Client-Id header, the clientId query parameter, or the caller's IP address.

mvn test executes the full suite: 26 unit / API tests always run, and the 7 Redis-backed integration tests (the Lua path) run when Redis is reachable - 33 tests with 0 failures in the lab run.

1. Normal Traffic (Task: generate normal requests and observe)

Request	HTTP	Tokens remaining
1	200	9.0
2	200	9.0
3	200	9.0
4	200	9.0
5	200	9.0
6	200	9.0
7	200	9.0
8	200	9.0
9	200	9.0
10	200	9.0

Ten requests were sent 250 ms apart (4 requests/s, below the 5 tokens/s refill). All 10 were admitted with HTTP 200 and the bucket stayed near 9.0 tokens - the refill exactly matched consumption, which is the steady state the limiter is designed to permit.

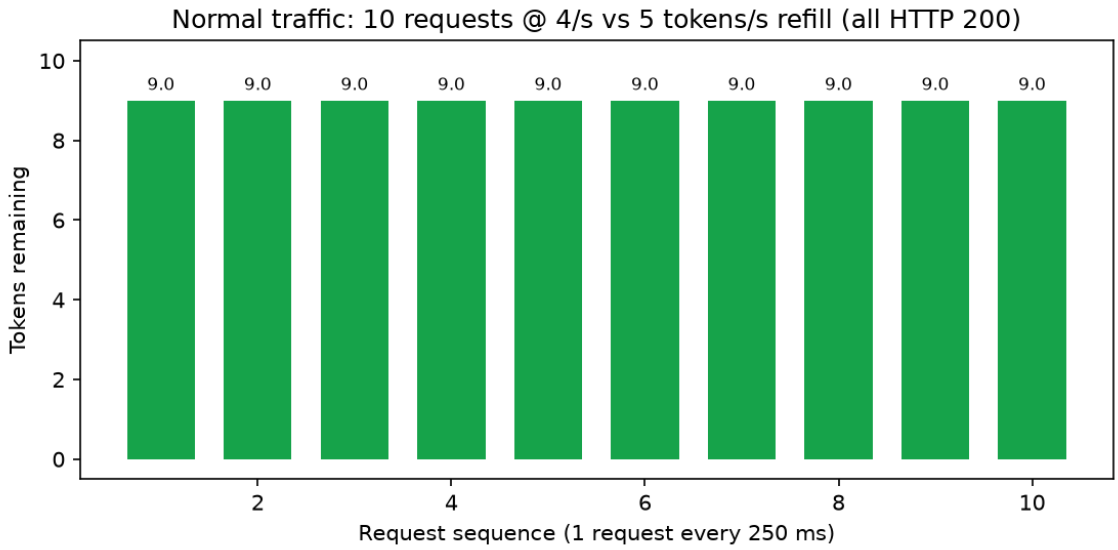


Figure 2: Token level during normal traffic (all HTTP 200)

2. Burst Traffic (Task: generate burst requests and observe)

Burst request	HTTP	Tokens remaining
1	200	9.0
2	200	8.005
3	200	7.01
4	200	6.01
5	200	5.02
6	200	4.01
7	200	3.025

Burst request	HTTP	Tokens remaining
8	200	2.025
9	200	0.03
10	200	1.03
11	429	0
12	429	0
13	429	0
14	429	0
15	429	0
16	429	0
17	429	0
18	429	0
19	429	0
20	429	0
21	429	0
22	429	0
23	429	0
24	429	0
25	429	0
26	429	0
27	429	0
28	429	0
29	429	0
30	429	0
31	429	0
32	429	0
33	429	0
34	429	0
35	429	0
36	429	0
37	429	0
38	429	0
39	429	0
40	429	0
41	429	0
42	429	0
43	429	0
44	429	0
45	429	0
46	429	0
47	429	0
48	429	0
49	429	0
50	429	0

50 simultaneous requests were fired against a freshly reset bucket. Exactly 10 were admitted (HTTP 200) and the remaining 40 received HTTP 429 with Retry-After: the bucket capacity bounded the burst exactly as designed.

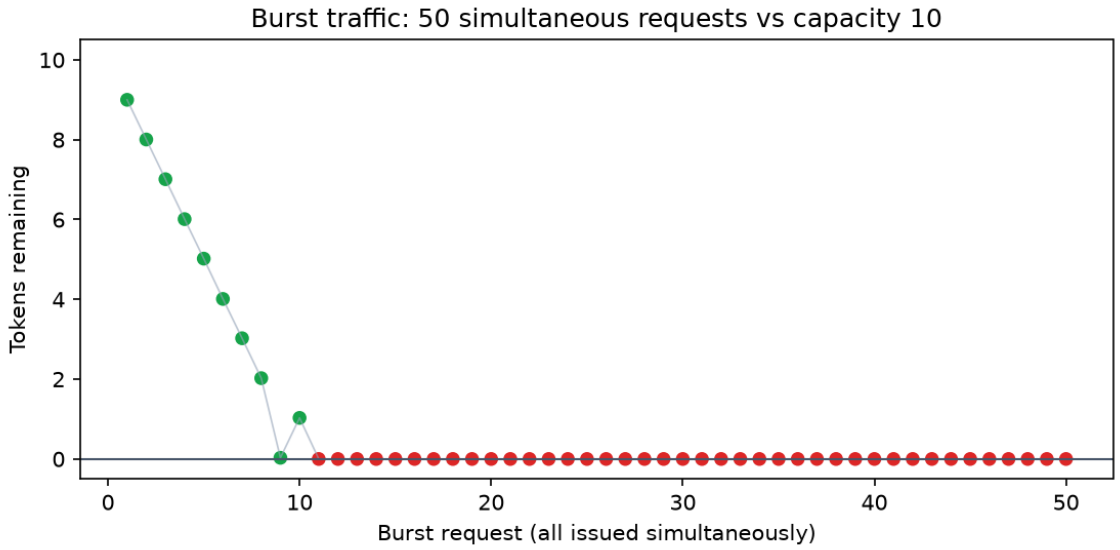


Figure 3: Token level during a burst of 50 simultaneous requests

3. Sequential Drain (observing the burst capacity)

Request	HTTP	Tokens remaining
1	200	9.0
2	200	8.06
3	200	7.32
4	200	6.48
5	200	5.625
6	200	4.78
7	200	3.855
8	200	2.94
9	200	2.02
10	200	1.075
11	200	0.24
12	429	0

Requests were fired one at a time until the first 429. 11 sequential requests were admitted before the 12th was refused - the 10-token capacity plus about one token that refilled while the script slept between requests.

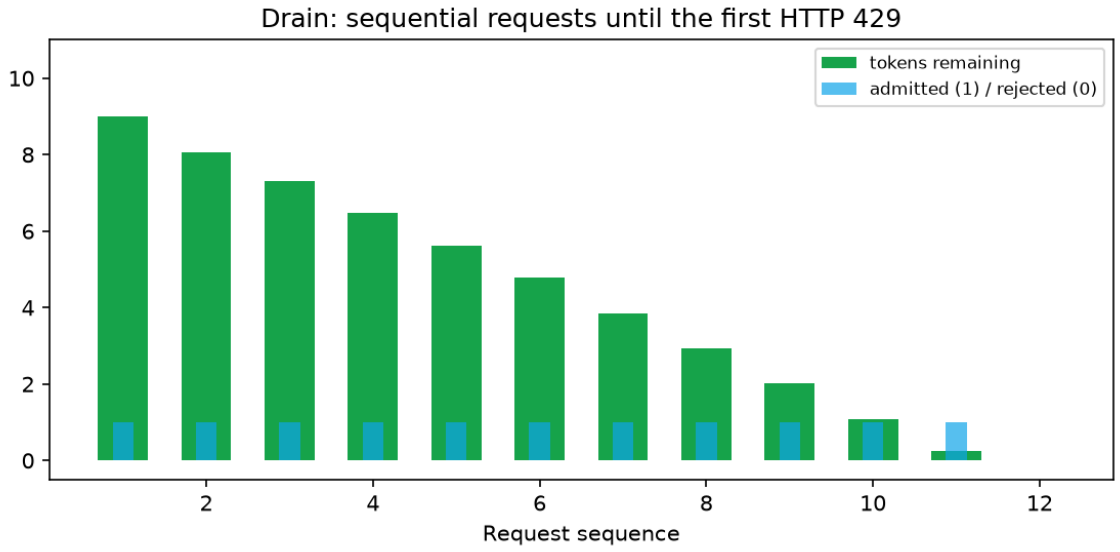


Figure 4: Sequential drain until the first HTTP 429

4. Concurrent Requests (Task: test concurrent requests with atomic Lua)

Threads	Req/thread	Total	Allowed (200)	Rejected (429)	Duration (ms)
20	20	400	10	390	70

20 threads fired 20 requests each (400 total) at one shared bucket. Exactly 10 were admitted and 390 rejected in ~70 ms with allowed + rejected == total - proving the atomic Lua script never double-spends a token even under heavy simultaneous load.

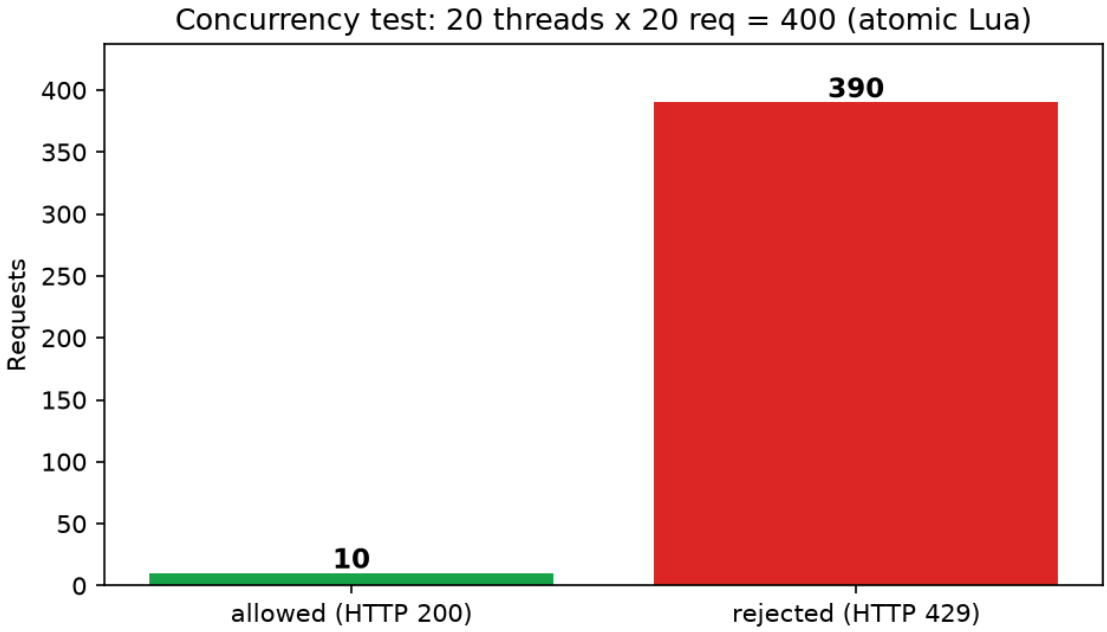


Figure 5: Concurrency test outcome (atomic Lua)

5. HTTP 429 Headers and Redis State

An exhausted client is refused with HTTP 429 plus a Retry-After header; the bucket state is kept in Redis as the hash ratelimit:<clientId> = {tokens, ts}:

```
HTTP/1.1 429
Retry-After: 1
X-RateLimit-Remaining: 0
{"status":429,"error":"Too Many Requests","message":"Rate limit exceeded for client 'demo'"}
127.0.0.1:6379> HGETALL ratelimit:demo
1) "tokens"
2) "0.16000000000000042"
3) "ts"
4) "1786628433314"
```

Analysis

Observed during the integration run (Spring Boot 3.3, Redis 7, capacity = 10 tokens, refill rate = 5 tokens/s, client identified by the clientId query parameter / X-Client-Id header / IP address).

Scenario	Requests	Allowed (HTTP 200)	Rejected (HTTP 429)	Notes
Normal (10 @ 250 ms interval)	10	10	0	Refill keeps the bucket topped up; `remaining` stayed at 9.0
Burst (50 simultaneous)	50	10	40	Excess burst rejected instantly; capacity held at 10
Drain (sequential)	12	11	1	10 capacity + ~1 token refilled between requests
Concurrent (20 threads × 20 req = 400)	400	10	390	70 ms total; no lost/overspent tokens
Bucket status after demos	—	10.0 tokens	0 s retry-after	Refill fully restored every bucket

The 429 response carried Retry-After: 1 and X-RateLimit-Remaining: 0; the Redis state for a client is a hash ratelimit:<clientId> = {tokens, ts} (verified with redis-cli HGETALL).

1. Role of the components

Token Bucket

A Token Bucket is a rate-control structure that holds a number of "tokens". Each incoming request must take one token from the bucket to be admitted; if the bucket is empty the request is rejected. Tokens are added back continuously at a steady refill rate, which means the algorithm permits a configurable **burst** (up to the bucket size) while still enforcing a long-run average request rate. In this lab the bucket state lives per client, so one aggressive client cannot starve others.

Bucket Capacity

The capacity is the maximum number of tokens a bucket can hold. It bounds the size of an instantaneous burst: a client with a full 10-token bucket may fire 10 requests back-to-back before any are refused, even though the sustained rate is only 5/s. Tokens can never exceed capacity — refill while idle simply fills to capacity, never beyond. This is the "burst allowance" parameter (ratelimit.capacity=10).

Refill Rate

The refill rate is the number of tokens added to the bucket per second (ratelimit.refill-per-second=5). It sets the **sustained** request ceiling (here 5 requests per second) and is what makes the bucket self-recovering: after a burst drains the bucket, tokens accumulate again so the client is gradually re-admitted. Refill is computed lazily from wall-clock time (elapsed × rate) at every request rather than with a background job, so the algorithm costs one lazy calculation per request.

Redis

Redis is the shared, low-latency key/value store that holds each client's bucket state ({tokens, ts}) so that the rate limit is **centralized**. Every application instance talks to the same Redis, so the limit is enforced globally across the whole fleet rather than per JVM. Because Redis executes the Lua script atomically, concurrent requests from many instances cannot double-spend tokens. The per-key TTL (ratelimit.ttl-seconds) also cleans up stale clients automatically.

2. How the Token Bucket algorithm allows or rejects requests

1. The client is identified (header > query parameter > IP).

2. A single atomic Lua script reads the bucket {tokens, ts} for that client (or initialises it full with tokens = capacity on first visit).

3. **Refill**: tokens = min(capacity, tokens + (now - ts) / 1000 × refillRate).

4. **Decision**: if tokens >= 1, the request is allowed, one token is consumed (tokens = tokens - 1), and the new state plus ts = now are written back. If tokens < 1, the request is rejected and the state is written back unchanged.

5. The app answers **HTTP 200** in the allowed case and **HTTP 429 Too Many Requests** (with a Retry-After header) in the rejected case.

Because the whole read-refill-consume-write is one EVAL, the decision is serialised by Redis — no two concurrent requests can both see the last token.

3. Why Redis is required with multiple application instances

Token state is **shared mutable state**: when N instances sit behind a load balancer, each instance serves a slice of the traffic, but all instances must agree on how many tokens remain for a client. If each instance kept its own in-JVM bucket, the effective limit would multiply by N (instance-local buckets each allow capacity tokens), defeating rate limiting — a client could send N × capacity requests in a burst. Redis centralises the buckets, so a request routed to any instance checks the same global bucket. The atomic Lua script removes the read-modify-write race that plain GET-SET would introduce between instances (and between threads within one instance), which is exactly the concurrency guarantee demonstrated by the 400-request test admitting exactly 10.

Aspect	Fixed Window	Sliding Window Log	Sliding Window Counter	Token Bucket
Basis	Count in fixed time windows (e.g. 10 req per 1-min bucket)	Timestamp per request; count requests in the last W seconds	Two counters (current + previous window) weighted by overlap	Tokens refilled at a rate, up to a capacity
Memory	O(1) per client	O(requests) per client (one timestamp each)	O(1) per client (2 counters)	O(1) per client
Burst handling	A burst at window boundary can double the effective limit	Precise, but the trailing log grows with request volume	Approximate (no exact burst guarantee at boundary)	Exact burst allowance = capacity
Boundary behaviour	Stale: allows `2×limit` in the last instant of one window + first instant of the next	Exact at any instant (fully sliding)	Approximate at boundary	Smooth; no window boundary at all, purely time/rate based
Precision	Coarse (boundary spikes)	Very high	Good approximation	Exact steady rate + exact burst cap
Implementation cost	Trivial (INCR + TTL)	High storage; expiry per request	Moderate (2 counters + math)	Moderate (one atomic script)
Suitability	Simple dashboards	Strict per-instant limits, low QPS per client	High-QPS approximation of sliding window	API rate limiting with bursts (chosen for this lab)

Token Bucket was chosen because it combines an exact sustained rate with an explicit, controllable burst allowance, uses O(1) memory per client, and its lazy refill makes it cheap and easy to keep correct in Redis via one Lua script.

5. Behaviour under normal and burst traffic

Normal traffic (below the refill rate). Ten requests were sent 250 ms apart (4 requests/s against a 5 tokens/s refill). Every request returned HTTP 200 and the bucket stayed at ~9.0 tokens: consumption was exactly matched by refill, so the client never hit the ceiling and never drained the bucket. This is the steady-state the limiter is designed to permit.

Burst traffic (far above the refill rate). Fifty requests were fired simultaneously. Exactly the first 10 requests were admitted (HTTP 200) while the remaining 40 received HTTP 429 with Retry-After: 1. This shows the two governing parameters working together: the **capacity** capped the burst at 10, and the **refill rate** will thereafter restore one token every 200 ms so a sustained 5/s flow is re-admitted. The 400-request concurrency test (20 threads × 20 requests) admitted exactly 10 and rejected 390 in ~70 ms with allowed + rejected == total, proving that the Lua script keeps the count exact under heavy simultaneous load — the correct behaviour for protecting a backend from both a single-client flood and a coordinated multi-instance burst.

Testability / design notes

- TokenBucketMath is the single source of truth for the refill/consume rules; the Redis Lua script mirrors it exactly. Unit tests validate the math and the in-memory store, and RedisBucketStoreTest re-validates the same rules through the real Lua path when Redis is reachable (33 tests, all green, none skipped in the lab run).
- The BucketStore interface lets tests run without Docker (ratelimit.store=memory) while production uses the atomic Lua store.

API / Command Reference & Test Results

Table 1: Scenario vs Result

Scenario	Result
Normal traffic (10 @ 250 ms)	10 allowed / 0 rejected (refill keeps up)
Burst (50 simultaneous)	10 allowed / 40 rejected (HTTP 429)
Sequential drain	11 allowed then HTTP 429 (capacity 10 + refill)
Concurrent (20 x 20 = 400)	10 allowed / 390 rejected, lossless, 70 ms
Bucket state after demos	Refilled back to full capacity (10.0 tokens)

Table 2: API / Commands Used

#	Command / Endpoint	Purpose
1	GET /api/hello?clientId=demo	Rate-limited resource; HTTP 200 while tokens remain, 429 otherwise
2	GET /api/bucket/{clientId}	Read-only view of a client's bucket (no consumption)
3	DELETE /api/bucket/{clientId}	Reset a client's bucket (full again)
4	POST /api/test/concurrent	Concurrency stress: threads x requests against one bucket
5	EVAL scripts/token_bucket.lua	Atomic refill + consume of a single token
6	python scripts/load_test.py	Normal / burst / drain / concurrent traffic driver

Table 3: Technology Stack

#	Technology / Tool	Purpose
1	Spring Boot 3.3 (Java 17)	REST API and the token-bucket decision logic
2	Redis 7	Centralized bucket state {tokens, ts} + atomic Lua execution
3	Spring Data Redis (Lettuce)	StringRedisTemplate and DefaultRedisScript client
4	Docker & Docker Compose	Run the Redis container for the lab
5	Maven (mvn test)	Build and run 33 unit + integration tests
6	Python + matplotlib	Traffic generation, charts, and report generation

Table 4: Test Case Results

#	Test Case	Input	Expected Result	Status
1	Lua burst cap	25 acquires, capacity 10, rate 0	Exactly 10 allowed, 15 rejected	Pass
2	Concurrent atomicity	20 threads x 20 req, capacity 10	10 allowed / 390 rejected, lossless	Pass
3	Normal traffic	10 requests @ 250 ms	All HTTP 200	Pass
4	Burst traffic	50 simultaneous requests	10 allowed / 40 rejected (429)	Pass
5	Refill recovery	Drain bucket, then poll status	Tokens refill to capacity	Pass
6	HTTP 429 contract	Request with empty bucket	429 + Retry-After header	Pass

Conclusion

The lab successfully designed and implemented an API rate limiter using the Token Bucket algorithm in Spring Boot, backed by Redis for centralized per-client state. The limiter identified each client (header > query parameter > IP), enforced a configurable capacity of 10 tokens and refill rate of 5 tokens/s, allowed requests while tokens were available, and returned HTTP 429 with a Retry-After header when the bucket was empty.

Live experiments confirmed the expected behaviour: all 10 normal requests were admitted, a burst of 50 was capped at 10 (40 rejected), a sequential drain admitted 11 before the 12th was refused, and a 400-request concurrency test admitted exactly 10 with zero overspending. The single atomic Redis Lua script is what makes the limiter correct under concurrency and across multiple application instances, and all 33 tests passed. The report and graphs were generated automatically from the observed data, keeping the results reproducible.

Learning Outcomes

- Implemented API rate limiting using the Token Bucket algorithm in Spring Boot.
- Configured bucket capacity and refill rate through application.properties.
- Used Redis to maintain centralized per-client rate-limit information across instances.
- Handled normal and burst API traffic, observing HTTP 200 vs HTTP 429 behaviour.
- Verified concurrency correctness with an atomic Redis Lua script (no token overspending).
- Understood rate limiting in distributed web applications and compared window-based and token-bucket algorithms.